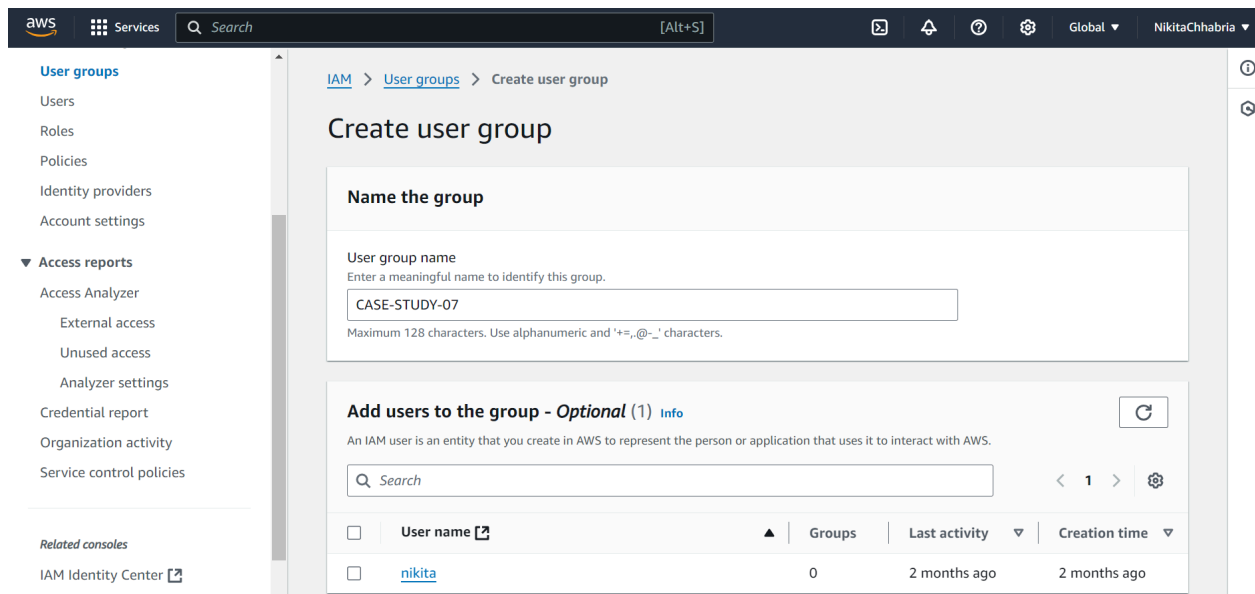


Case Study 7: Kubernetes Cluster Management with Terraform

- **Concepts Used:** Kubernetes, Terraform, AWS Cloud9.
- **Problem Statement:** "Use Terraform to provision a Kubernetes cluster on AWS. Then, use AWS Cloud9 IDE to deploy a sample application on the cluster using kubectl."
- **Tasks:**
 - Write a Terraform script to create a Kubernetes cluster on AWS.
 - Use AWS Cloud9 to configure kubectl for the newly created cluster.
 - Deploy a simple application (e.g., a Python Flask app) on the Kubernetes cluster and verify its deployment.

Step 1: Create IAM user and user group



AWS Managed Policies:

- 1) AmazonEKSClusterPolicy
- 2) AmazonEKSWorkerNodePolicy
- 3) AmazonVPCFullAccess
- 4) IAMFullAccess

Customer Managed Policies:

- 1) DescribeInstance

JSON:

```
{  
  "Version": "2012-10-17",
```

```
        "Statement": [
            {
                "Effect": "Allow",
                "Action": [
                    "ec2:DescribeInstanceCreditSpecifications",
                    "ec2:DescribeInstances",
                    "ec2:DescribeInstanceAttribute"
                ],
                "Resource": "*"
            }
        ]
    }
}

2) EKSNodeGroup
JSON:
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "eks:CreateNodegroup",
                "eks:DescribeNodegroup",
                "eks>DeleteNodegroup",
                "eks:CreateCluster",
                "eks:DescribeCluster",
                "eks>DeleteCluster",
                "iam:PassRole"
            ],
            "Resource": "*"
        }
    ]
}
```

3) EKSPullAccess

JSON

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "eks:CreateCluster",
                "eks:DescribeCluster",
                "eks:ListClusters",
```

```

        "eks:DeleteCluster",
        "ec2:DescribeSubnets",
        "ec2:DescribeVpcs",
        "iam:CreateServiceLinkedRole",
        "iam:PassRole"
    ],
    "Resource": "*"
}
]
}

```

4) InstancePolicy

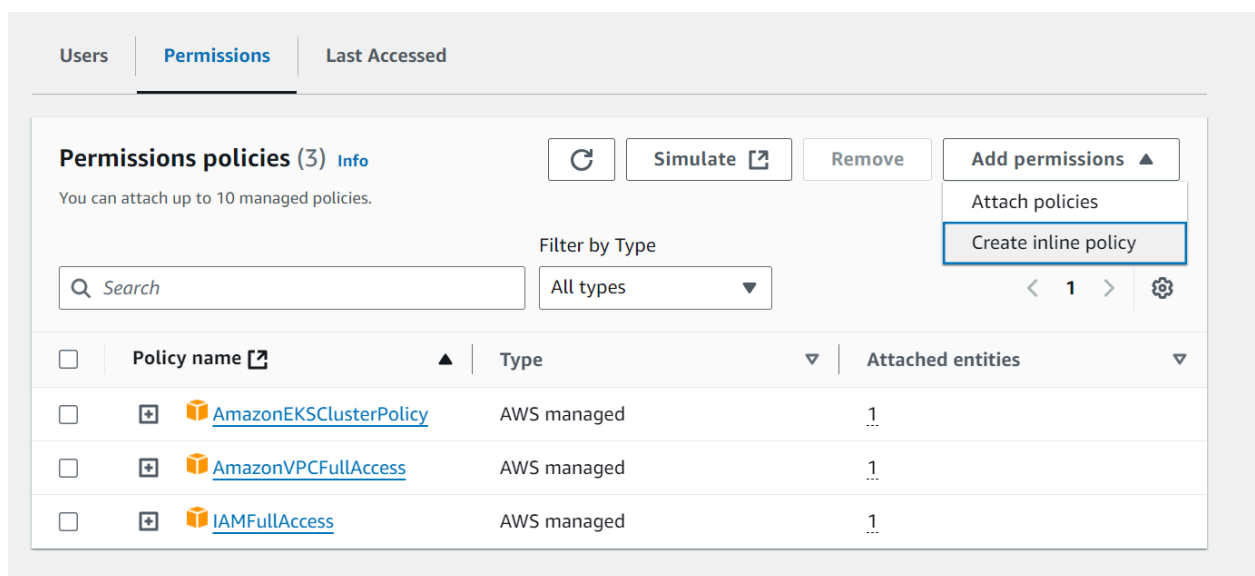
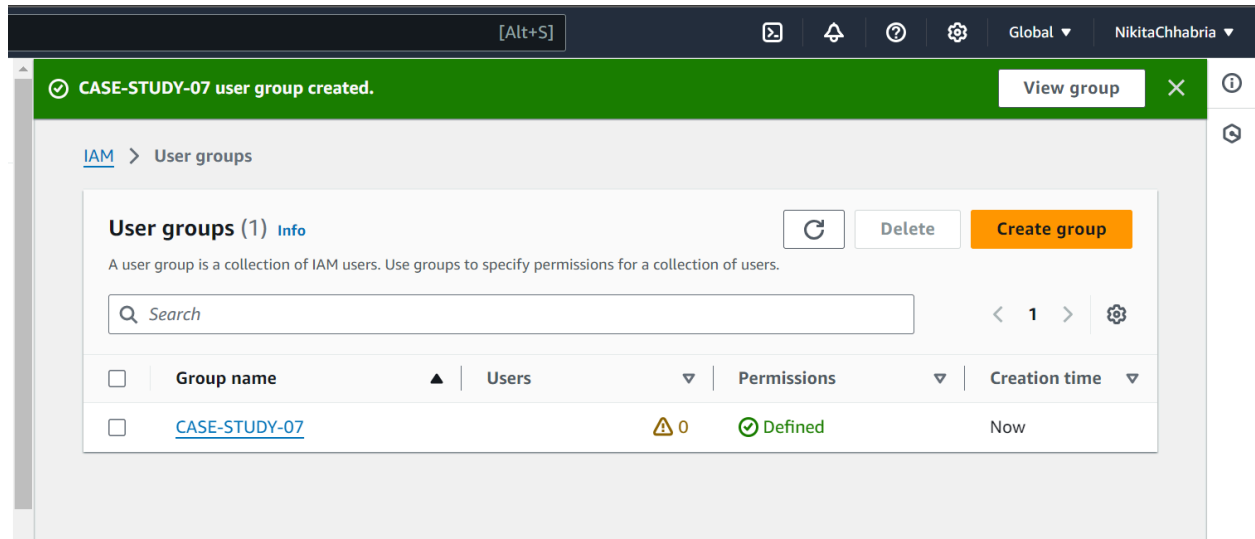
JSON:

```

{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "ec2:RunInstances",
                "ec2:DescribeInstances",
                "ec2:TerminateInstances",
                "ec2:CreateTags",
                "ec2:DescribeKeyPairs"
            ],
            "Resource": "*"
        }
    ]
}

```

The customer managed policies can be added easily while configuring the user group for the customer managed policies. After the creation of the user group, Go to Permission and then add permissions. These permissions are to be added using inline json code.



Make sure to create new statement for every policy and give it unique names as they are to be used later

[IAM](#) > [User groups](#) > [CASE-STUDY-07](#) > Create policy

Step 1

Specify permissions

Step 2

Review and create

Specify permissions

Info

Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.

Policy editor

VisualJSONActions

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Sid": "Statement1",
6       "Effect": "Allow",
7       "Action": [],
8       "Resource": []
9     }
10  ]
11 }
```

Edit statement

Select a statement

Select an existing statement in the policy or add a new statement.

+ Add new statement

Policy DescribeInstance created.

User group name	Creation time	ARN
CASE-STUDY-07	October 21, 2024, 11:52 (UTC+05:30)	arn:aws:iam::026090558619:group/CASE-STUDY-07

UsersPermissionsLast Accessed

Permissions policies (4)

Info

You can attach up to 10 managed policies.

Filter by Type

Search

All types

1

Policy name	Type	Attached entities
AmazonEKSClusterPolicy	AWS managed	1

Policy editor

VisualJSONActions

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": [
7         "eks:CreateNodegroup",
8         "eks:DescribeNodegroup",
9         "eks>DeleteNodegroup",
10        "eks:CreateCluster",
11        "eks:DescribeCluster",
12        "eks>DeleteCluster",
13        "iam:PassRole"
14      ],
15      "Resource": "*"
16    }
17  ]
18 }
19 }
```

Edit statement

Select a statement

Select an existing statement in the policy or add a new statement.

+ Add new statement

Identity and Access Management (IAM)

Search IAM

Dashboard

Access management

- User groups
- Users
- Roles
- Policies
- Identity providers
- Account settings

Access reports

- Access Analyzer

Policy EKSNodeGroup created.

CASE-STUDY-07October 21, 2024, 11:52 (UTC+05:30)arn:aws:iam::026090558619:group/CASE-STUDY-07

UsersPermissionsLast Accessed

Permissions policies (5) Info

You can attach up to 10 managed policies.

SearchFilter by TypeAll types

☐	Policy name	Type	Attached entities
☐	AmazonEKSClusterPolicy	AWS managed	1
☐	AmazonVPCFullAccess	AWS managed	1

Just like that all the policies are added

Permissions policies (7) [Info](#)

[Refresh](#) [Simulate](#) [Remove](#) [Add permissions](#)

You can attach up to 10 managed policies.

Filter by Type All types < 1 > [Settings](#)

☐	Policy name ↗	Type	Attached entities
☐	AmazonEKSClusterPolicy	AWS managed	1
☐	AmazonVPCFullAccess	AWS managed	1
☐	DescribeInstance	Customer inline	0
☐	EKSFullAccess	Customer inline	0
☐	EKSTaskRole	Customer inline	0
☐	IAMFullAccess	AWS managed	1
☐	InstanceProfile	Customer inline	0

After this is added, Go to users and add users> During security make sure to add this in the user group we created above

[IAM](#) > [Users](#) > Create user

Step 1
Specify user details

Step 2
Set permissions

Step 3
Review and create

Specify user details

User details

User name

NIKITA07

The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and + = , . @ _ - (hyphen)

☐ Provide user access to the AWS Management Console - *optional*
If you're providing console access to a person, it's a [best practice](#) to manage their access in IAM Identity Center.

[?](#)

If you are creating programmatic access through access keys or service-specific credentials for AWS CodeCommit or Amazon Keyspaces, you can generate them after you create this IAM user. [Learn more](#)

[Cancel](#) [Next](#)

Step 2
Set permissions

Step 3
Review and create

functions. [Learn more](#)

Permissions options

☒ **Add user to group**
Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.

☐ **Copy permissions**
Copy all group memberships, attached managed policies, and inline policies from an existing user.

☐ **Attach policies directly**
Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

User groups (1/1)

< 1 > ⚙️

☒	Group name ↗	Users	Attached policies ↗	Created
☒	CASE-STUDY-07	0	IAMFullAccess , Amaz...	2024-10-21 (2...

Identity and Access Management (IAM)

- Dashboard
- ▼ Access management
 - User groups
 - Users**
 - Roles
 - Policies
 - Identity providers
 - Account settings
- ▼ Access reports
 - Access Analyzer
 - External access

✔ **User created successfully** View user ✕

You can view and download the user's password and email instructions for signing in to the AWS Management Console.

[IAM](#) > Users

Users (2) [Info](#)

< 1 > ⚙️

☐	User name	Path	Group	Last activity	MFA	Pa
☐	nikita	/	0	✔ 72 days ago	-	✔
☐	NIKITA07	/	0	-	-	-

Now, from here we need the access key as well as the password to configure aws with terraform. For that,

Go to the user created and the under security tab

The screenshot shows the AWS IAM console interface. On the left is a navigation sidebar with 'Identity and Access Management (IAM)' selected. The main content area shows the details for user 'NIKITA07'. The 'Security credentials' tab is active, displaying a 'Summary' section with fields for ARN, Console access (Disabled), Access key 1 (with a 'Create access key' link), Created date, and Last console sign-in. Below this is a table with tabs for 'Permissions', 'Groups (1)', 'Tags', 'Security credentials', and 'Last Accessed'. The 'Console sign-in' section is visible, showing a 'Console sign-in link' and a 'Console password' field, with an 'Enable console access' button.

You will find the option to create access keys

This screenshot shows the 'Access keys' section of the AWS IAM console for user 'NIKITA07'. It features a message about MFA devices and a 'Create access key' button. Below this, a section titled 'Access keys (0)' explains that users can have a maximum of two access keys at a time and provides a 'Create access key' button. At the bottom, there is a section for 'SSH public keys for AWS CodeCommit (0)' with an 'Upload SSH public key' button. The interface includes a top navigation bar with the AWS logo, a search bar, and a user profile dropdown.

We need the access key for configuring AWS CLI therefore choose that option

The screenshot shows the AWS IAM console interface for creating an access key for user 'NIKITA07'. The breadcrumb trail is 'IAM > Users > NIKITA07 > Create access key'. The left sidebar shows the progress: Step 1 (selected), Step 2 - optional, and Step 3. The main content area is titled 'Access key best practices & alternatives' with an 'Info' icon. It includes a warning about long-term credentials and a 'Use case' section with four radio button options: 'Command Line Interface (CLI)' (selected), 'Local code', 'Application running on an AWS compute service', and 'Third-party service'. Each option has a brief description of its use case.

Give it a description

This screenshot shows Step 2 of the 'Create access key' process, titled 'Set description tag - optional' with an 'Info' icon. The breadcrumb trail remains 'IAM > Users > NIKITA07 > Create access key'. The left sidebar indicates Step 2 is the current step. The main content area explains that the description tag will be attached to the user. A text input field is provided for the 'Description tag value', with the example 'Adv.Devops Practical Exam' entered. Below the field, a note states: 'Maximum 256 characters. Allowed characters are letters, numbers, spaces representable in UTF-8, and: _ . : / = + - @'. At the bottom right, there are three buttons: 'Cancel', 'Previous', and 'Create access key'.

Click on create and download the .csv file with the access key and password

Access key created
This is the only time that the secret access key can be viewed or downloaded. You cannot recover it later. However, you can create a new access key any time.

[IAM](#) > [Users](#) > [NIKITA07](#) > Create access key

Step 1
[Access key best practices & alternatives](#)

Step 2 - optional
[Set description tag](#)

Step 3
Retrieve access keys

Retrieve access keys [Info](#)

Access key
If you lose or forget your secret access key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.

Access key	Secret access key
AKIAQMEY6KSN43OUKXAY	***** Show

Access key best practices

- Never store your access key in plain text, in a code repository, or in code.

Step 2: Creation of EKS cluster using terraform

To set up terraform, make sure your system has terraform installed and the path is added to access it from anywhere on your device.

Scripts of terraform are written in json or hcl format. They have .tf extension to them. In my case, I am using vs code to write my scripts

The code written is taken from official documentation available on terraform hub. Add your credentials and region to the provided code

```
1 provider "aws" {
2   region = var.location
3 }
4
5 resource "aws_instance" "demo_server" {
6   ami           = var.ami_name
7   key_name      = var.key
8   instance_type = var.instance_type
9   associate_public_ip_address = true
10  subnet_id     = aws_subnet.demo_subnet_1.id
11  vpc_security_group_ids = [aws_security_group.demo_vpc_sg.id]
12  tags = {
13    Name = "demo-server-instance"
14  }
15 }
16
17 // Create VPC
18 resource "aws_vpc" "demo_vpc" {
19   cidr_block = var.vpc_cidr
20 }
21
```

Install aws cli on your terminal and make sure it is running by running the command “aws –version”. After that is confirmed run the command “aws configure” and fill in proper details

```
PS C:\Users\User\Downloads\AdvDevOpsPrac-main\AdvDevOpsPrac-main> aws --version
aws-cli/2.18.10 Python/3.12.6 Windows/11 exe/AMD64

PS C:\Users\User\Downloads\AdvDevOpsPrac-main\AdvDevOpsPrac-main> aws configure
AWS Access Key ID [None]: AKIAQMEY6KSN43OUKXAY
AWS Secret Access Key [None]: ^VAWS Secret Access Key [None]: AWS Secret Access Key [None]: ^V
Default region name [None]: us-east-1
Default output format [None]: json
```

Now that the system is setup we can finally initialize terraform

```
PS C:\Users\User\Downloads\AdvDevOpsPrac-main\AdvDevOpsPrac-main> terraform init
Initializing the backend...
Initializing modules...
- eks in eks
- sgs in sg_eks
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v5.72.1...
- Installed hashicorp/aws v5.72.1 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to update Terraform's configuration. If you forget, other
commands will tell you to do so if necessary.
```

After initialization, We need to run the command terraform plan

```
PS C:\Users\User\Downloads\casedtudy_07> terraform plan
var.key_name
  Enter a value: nikita_07

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
+ create

Terraform will perform the following actions:

# aws_instance.prac07 will be created
+ resource "aws_instance" "prac07" {
  + ami                        = "ami-06b21ccaeff8cd686"
  + arn                      = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone         = (known after apply)
  + cpu_core_count            = (known after apply)
  + cpu_threads_per_core      = (known after apply)
}
```

```
# tls_private_key.rsa_4096 will be created
+ resource "tls_private_key" "rsa_4096" {
  + algorithm                = "RSA"
  + ecdsa_curve              = "P224"
  + id                      = (known after apply)
  + private_key_openssh      = (sensitive value)
  + private_key_pem          = (sensitive value)
  + private_key_pem_pkcs8    = (sensitive value)
  + public_key_fingerprint_md5 = (known after apply)
  + public_key_fingerprint_sha256 = (known after apply)
  + public_key_openssh       = (known after apply)
  + public_key_pem           = (known after apply)
  + rsa_bits                 = 4096
}
```

Plan: 4 to add, 0 to change, 0 to destroy.

Now that all the resources are ready, we can finally apply the changes by terraform apply

```
PS C:\Users\User\Downloads\casedtudy_07> terraform apply
var.key_name
  Enter a value: niks_07

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
+ create

Terraform will perform the following actions:
```

```
Plan: 4 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

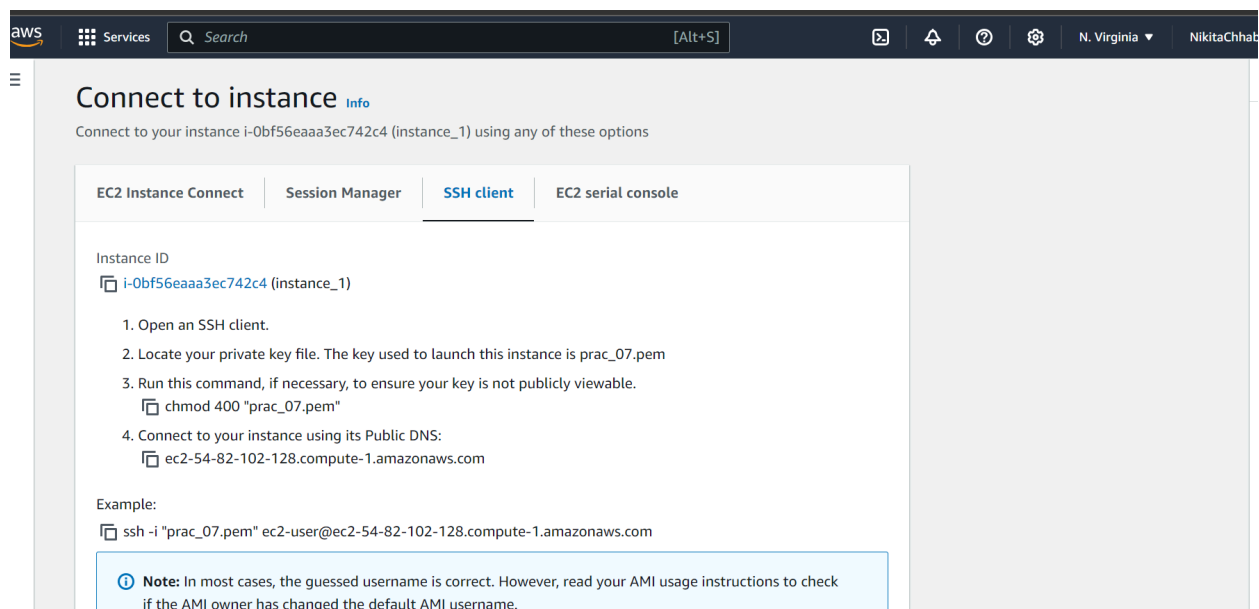
tls_private_key.rsa_4096: Creating...
tls_private_key.rsa_4096: Creation complete after 4s [id=116b4aedbe1b91fc02e4918540362c8e3d1dbff0]
aws_key_pair.key_pair: Creating...
local_file.private_key: Creating...
local_file.private_key: Creation complete after 0s [id=fb6077c304fd35ffeb94855a8bbd46bd2745f784]
aws_key_pair.key_pair: Creation complete after 1s [id=niks_07]
aws_instance.prac07: Creating...
aws_instance.prac07: Still creating... [10s elapsed]
aws_instance.prac07: Creation complete after 16s [id=i-060d1e58906476482]

Apply complete! Resources: 4 added, 0 changed, 0 destroyed.
```

Now if you go to your AWS console, you will see the EC2 instances and well as the Cluster created successfully.

Step 3: Deploy using Kubernetes

Now connect your EC2 through SSH client. I used git bash to do this, you can use command prompt as well



aws Services Search [Alt+S] N. Virginia NikitaChhat

Connect to instance [Info](#)

Connect to your instance i-0bf56eaaa3ec742c4 (instance_1) using any of these options

- EC2 Instance Connect
- Session Manager
- SSH client**
- EC2 serial console

Instance ID
i-0bf56eaaa3ec742c4 (instance_1)

1. Open an SSH client.
2. Locate your private key file. The key used to launch this instance is prac_07.pem
3. Run this command, if necessary, to ensure your key is not publicly viewable.
chmod 400 "prac_07.pem"
4. Connect to your instance using its Public DNS:
ec2-54-82-102-128.compute-1.amazonaws.com

Example:
ssh -i "prac_07.pem" ec2-user@ec2-54-82-102-128.compute-1.amazonaws.com

Note: In most cases, the guessed username is correct. However, read your AMI usage instructions to check if the AMI owner has changed the default AMI username.

```

User@DESKTOP-L7EPOIV MINGW64 ~/Downloads
$ ssh -i "casestudy.pem" ec2-user@ec2-18-208-144-127.compute-1.amazonaws.com
The authenticity of host 'ec2-18-208-144-127.compute-1.amazonaws.com (18.208.144.127)' can't be established.
ED25519 key fingerprint is SHA256:wVu08wj3+bBYm+QAMG9x8EOKTVzLWkdy8FFrEjnWwEk.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-18-208-144-127.compute-1.amazonaws.com' (ED25519) to the list of known hosts.

#####
~\      #####      Amazon Linux 2023
~~~    \#####/
~~~     \###/
~~~      \#/
~~~       V~'-----> https://aws.amazon.com/linux/amazon-linux-2023
~~~      /m/
~~~     /m/

[ec2-user@ip-172-31-95-138 ~]$

```

Make sure to download all the dependencies

```

[ec2-user@ip-172-31-95-138 ~]$ sudo yum update -y
Last metadata expiration check: 0:03:23 ago on Mon Oct 21 23:43:53 2024.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-95-138 ~]$ sudo yum install aws-cli
Last metadata expiration check: 0:03:43 ago on Mon Oct 21 23:43:53 2024.
Package awscli-2-2.15.30-1.amzn2023.0.1.noarch is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-95-138 ~]$ |

```

```

[ec2-user@ip-172-31-95-138 ~]$ sudo yum install docker -y
Last metadata expiration check: 0:04:23 ago on Mon Oct 21 23:43:53 2024.
Dependencies resolved.
=====
Package                                Arch      Version                                Repository    Size
=====
Installing:
docker                                x86_64    25.0.6-1.amzn2023.0.2                amazonlinux    44 M
Installing dependencies:
containerd                            x86_64    1.7.22-1.amzn2023.0.2                amazonlinux    36 M
iptables-libs                         x86_64    1.8.8-3.amzn2023.0.2                amazonlinux    401 k
iptables-nft                          x86_64    1.8.8-3.amzn2023.0.2                amazonlinux    183 k
libcgroup                             x86_64    3.0-1.amzn2023.0.1                  amazonlinux    75 k
libnetfilter_conntrack                x86_64    1.0.8-2.amzn2023.0.2                amazonlinux    58 k
libnfnetlink                          x86_64    1.0.1-19.amzn2023.0.2               amazonlinux    30 k
libnftnl                              x86_64    1.2.2-2.amzn2023.0.2                amazonlinux    84 k
pigz                                  x86_64    2.5-1.amzn2023.0.3                  amazonlinux    83 k
runc                                  x86_64    1.1.14-1.amzn2023.0.1               amazonlinux    3.2 M
=====
Transaction Summary

```

```

[ec2-user@ip-172-31-95-138 ~]$ sudo service docker start
Redirecting to /bin/systemctl start docker.service
[ec2-user@ip-172-31-95-138 ~]$ sudo usermod -a -G docker ec2-user
[ec2-user@ip-172-31-95-138 ~]$ |

```

```
[ec2-user@ip-172-31-95-138 ~]$ curl -LO "https://dl.k8s.io/release/${curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100 138    100 138    0     0  2562      0 --:--:-- --:--:-- --:--:-- 2603
100 53.7M 100 53.7M    0     0  56.3M      0 --:--:-- --:--:-- --:--:-- 58.3M
[ec2-user@ip-172-31-95-138 ~]$ file kubectl
kubectl: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), statically linked, Go BuildID=HbeA0A2ZbqMvqp
as7Me3/VXj1w6jygKKw_b8pTijn/xwTlUXp2PSPa9Uaz1tuM/1Doon0MdqFPszaPMmL9F, stripped
[ec2-user@ip-172-31-95-138 ~]$ chmod +x kubectl
[ec2-user@ip-172-31-95-138 ~]$ sudo mv kubectl /usr/local/bin/kubectl
[ec2-user@ip-172-31-95-138 ~]$ kubectl version --client
Client Version: v1.31.1
Kustomize Version: v5.4.2
[ec2-user@ip-172-31-95-138 ~]$ |
```

```
[ec2-user@ip-172-31-95-138 ~]$ aws configure
AWS Access Key ID [None]: AKIAQMEY6KSN43OUKXAY
AWS Secret Access Key [None]: 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
Default region name [None]: us-east-1
Default output format [None]: json
```

```
[ec2-user@ip-172-31-95-138 ~]$ aws eks list-clusters --region us-east-1
{
  "clusters": [
    "casestudy07"
  ]
}
[ec2-user@ip-172-31-95-138 ~]$ aws eks update-kubeconfig --region us-east-1 --name casestudy07
Added new context arn:aws:eks:us-east-1:026090558619:cluster/casestudy07 to /home/ec2-user/.kube/config
```

After that i done, make a folder for your application and write the code using nano

```
[ec2-user@ip-172-31-95-138 ~]$ mkdir flask_app
[ec2-user@ip-172-31-95-138 ~]$ cd flask_app
[ec2-user@ip-172-31-95-138 flask_app]$
```

```
[ec2-user@ip-172-31-95-138 flask_app]$ nano app.py
```

```
GNU nano 3.8 app.py
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello I am Nikita Chhabria, This is case study 07'

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=80)
```

Make sure to install docker to create the image of the application on your account

```
[ec2-user@ip-172-31-95-138 flask_app]$ docker build -t flask-app:latest .
[+] Building 7.5s (10/10) FINISHED
=> [internal] load build definition from Dockerfile                                docker:default 0.0s
=> => transferring dockerfile: 500B                                              0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim                0.4s
=> [internal] load .dockerignore                                                  0.0s
=> => transferring context: 2B                                                    0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:7a9cd42706c174cdc         3.3s
=> => resolve docker.io/library/python:3.9-slim@sha256:7a9cd42706c174cdc         0.0s
=> => sha256:e9cf9c2f800532238969770769696b30e2b270f3628 1.75kB / 1.75kB       0.0s
=> => sha256:b9b3c02da6c33a199501e9e4cf8da859d8065718b08 5.43kB / 5.43kB       0.0s
=> => sha256:a480a496ba95a197d587aad9e0f545ca7dbd4049 29.13MB / 29.13MB        0.9s
=> => sha256:99b8d55c8acd10aa3901ad6f43d5998b882c1f4acac 3.51MB / 3.51MB        0.4s
=> => sha256:151089ffef3f9093a349049321fa9a4668c29b122 14.92MB / 14.92MB        0.8s
=> => sha256:7a9cd42706c174cdc5f78880ab9ae3b6551323a7d 10.41kB / 10.41kB       0.0s
=> => sha256:277f520eee4a7406e307add15a461fa57bfe184f595671f 250B / 250B        0.5s
=> => extracting sha256:a480a496ba95a197d587aad9e0f545ca7dbd40495a47153 1.3s
=> => extracting sha256:99b8d55c8acd10aa3901ad6f43d5998b882c1f4acac51f0 0.1s
=> => extracting sha256:151089ffef3f9093a349049321fa9a4668c29b122d05224e 0.6s
=> => extracting sha256:277f520eee4a7406e307add15a461fa57bfe184f595671f3 0.0s
=> [internal] load build context                                                  0.0s
=> => transferring context: 443B                                                  0.0s
=> [2/5] WORKDIR /app                                                            0.2s
=> [3/5] COPY requirements.txt requirements.txt                                  0.0s
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt                     3.3s
=> [5/5] COPY app.py app.py                                                      0.0s
=> exporting to image                                                            0.2s
=> => exporting layers                                                            0.2s
=> => writing image sha256:3ec59a6de3ec904f2d9b0255c2397aaa2da2f5b73ef26 0.0s
=> => naming to docker.io/library/flask-app:latest                             0.0s
[ec2-user@ip-172-31-95-138 flask_app]$
```

Login on docker with the console

```
[ec2-user@ip-172-31-95-138 flask_app]$ docker login
Log in with your Docker ID or email address to push and pull images from Docker Hub. If you don't have a Docker ID, head over to https://docs.docker.com/get-docker/. You can log in with your password or a Personal Access Token (PAT). Using a limited-scope PAT grants better security and is recommended for most users. See https://docs.docker.com/go/access-tokens/ for more.

Username: nikitachhabria28
Password:
WARNING! Your password will be stored unencrypted in /home/ec2-user/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
```

Tag and push the code on docker

```
[ec2-user@ip-172-31-95-138 flask_app]$ docker tag flask-app:latest nikitachhabria28/flask-app:latest
[ec2-user@ip-172-31-95-138 flask_app]$ docker push nikitachhabria28/flask-app:latest
The push refers to repository [docker.io/nikitachhabria28/flask-app]
6c1038fd74fa: Pushed
c60762c2b596: Pushed
07fc6fb93ccb: Pushed
107d42bf7caf: Pushed
d86feaf80e98: Pushed
19f5accf4683: Pushed
0300a07ea341: Pushed
98b5f35ea9d3: Pushed
latest: digest: sha256:e475f72ecab9d39dc428acba2e69e2a49365cad610f012e8c1bc14ce7144a size: 1990
```

Now, create a Dockerfile that would guide docker on how to run the commands and in what order.

nano Dockerfile

Paste the following code

```
# Use the official Python image from the Docker Hub  
FROM python:3.9-slim
```

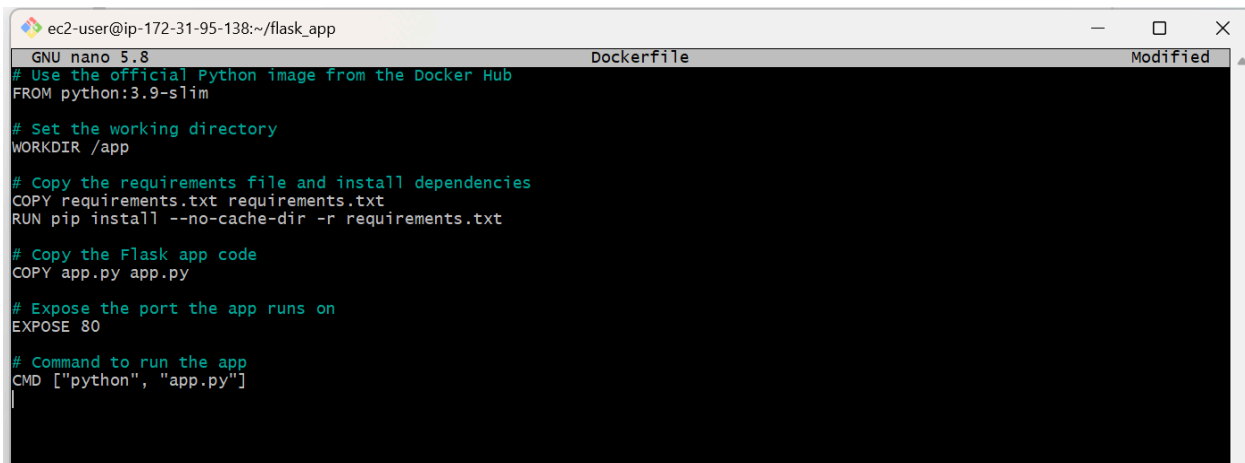
```
# Set the working directory  
WORKDIR /app
```

```
# Copy the requirements file and install dependencies  
COPY requirements.txt requirements.txt  
RUN pip install --no-cache-dir -r requirements.txt
```

```
# Copy the Flask app code  
COPY app.py app.py
```

```
# Expose the port the app runs on  
EXPOSE 80
```

```
# Command to run the app
```

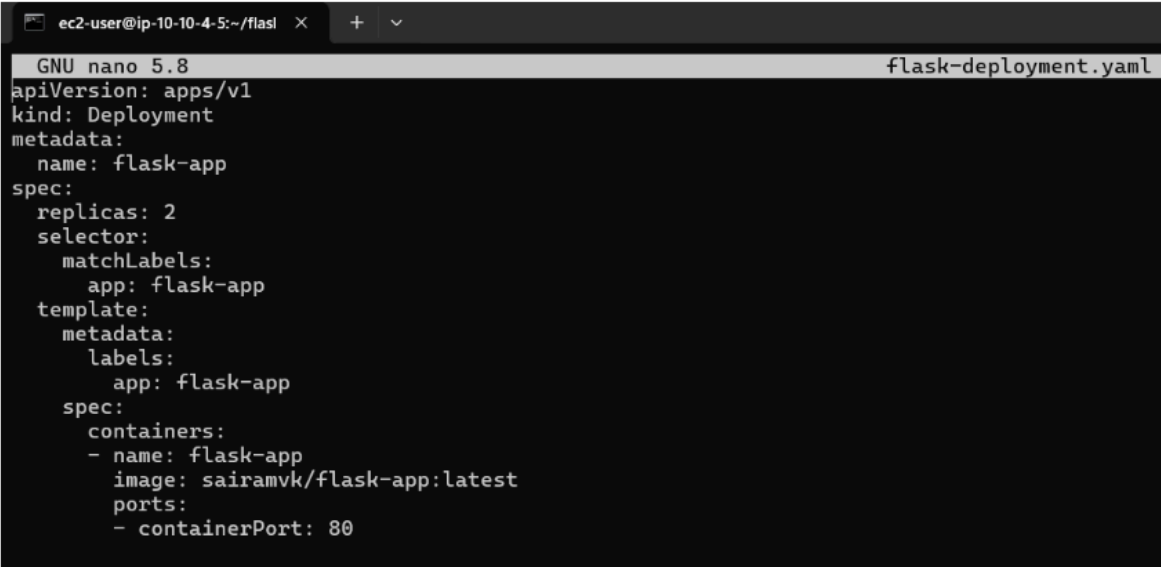
A screenshot of a terminal window titled 'ec2-user@ip-172-31-95-138:~/flask_app'. The terminal shows the content of a Dockerfile being edited in nano 5.8. The Dockerfile content is as follows:

```
GNU nano 5.8 Dockerfile Modified  
# Use the official Python image from the Docker Hub  
FROM python:3.9-slim  
  
# Set the working directory  
WORKDIR /app  
  
# Copy the requirements file and install dependencies  
COPY requirements.txt requirements.txt  
RUN pip install --no-cache-dir -r requirements.txt  
  
# Copy the Flask app code  
COPY app.py app.py  
  
# Expose the port the app runs on  
EXPOSE 80  
  
# Command to run the app  
CMD ["python", "app.py"]
```

- 1) Now, deploy the application to kubernetes. Create flask-deployment.yaml file using **nano flask-deployment.yaml** and add the following content.

```
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: flask-app  
spec:
```

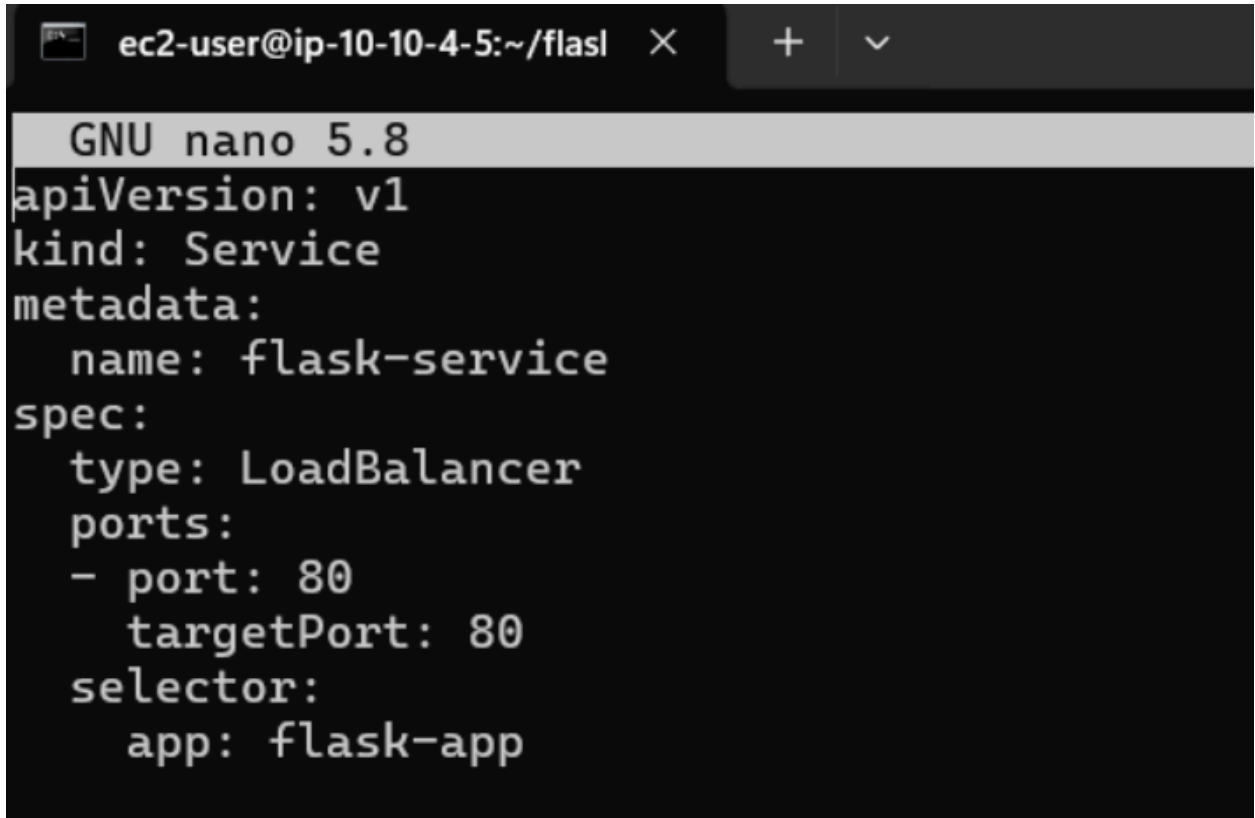
```
replicas: 2
selector:
  matchLabels:
    app: flask-app
template:
  metadata:
    labels:
      app: flask-app
spec:
  containers:
  - name: flask-app
    image: <your-dockerhub-username>/flask-app:latest
    ports:
      - containerPort: 80
```

A screenshot of a terminal window with a dark background. The title bar shows 'ec2-user@ip-10-10-4-5: ~/flaskl' and a tab labeled 'flask-deployment.yaml'. The terminal content shows the nano editor's header 'GNU nano 5.8' and the following YAML code:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: flask-app
  template:
    metadata:
      labels:
        app: flask-app
    spec:
      containers:
      - name: flask-app
        image: sairamvk/flask-app:latest
        ports:
          - containerPort: 80
```

- 1) Apply the created deployment
kubectl apply -f flask-deployment.yaml
- 1) To expose this application, we need to create a flask-service.yaml file. Use **nano flask-service.yaml** and add the following code.
apiVersion: v1
kind: Service
metadata:
 name: flask-service
spec:
 type: LoadBalancer

```
ports:
- port: 80
  targetPort: 80
selector:
  app: flask-app
```

A screenshot of a terminal window. The title bar shows 'ec2-user@ip-10-10-4-5:~/flas' with window control buttons. The terminal content shows the 'GNU nano 5.8' prompt followed by a YAML configuration for a Kubernetes Service. The configuration includes 'apiVersion: v1', 'kind: Service', 'metadata: name: flask-service', and 'spec: type: LoadBalancer' with a 'ports' section containing 'port: 80' and 'targetPort: 80', and a 'selector' section with 'app: flask-app'.

```
GNU nano 5.8
apiVersion: v1
kind: Service
metadata:
  name: flask-service
spec:
  type: LoadBalancer
  ports:
  - port: 80
    targetPort: 80
  selector:
    app: flask-app
```

Access this link on a browser and check this.



Hello this is Nikita Chhabria and this is case study 07